CODEMIGRATE · PROJECT REVIEW

Team Guide: who does what, and how it all works

A simple explanation of the whole project, split into five parts, one per team member. Each part has: what you own, an easy way to explain it, the code to show, the numbers to remember, likely reviewer questions with answers, and the paper section you write.

> **Important:** this guide is for **understanding and preparing**. Do not copy its sentences into the IEEE paper. The paper must be under 10% AI-generated and under 10% similarity, so write your section in your own words from the code and the results.

Contents

- 1. The project in one minute
- 2. Architecture (diagram + one example walked through)
- 3. Task split for 5 members (overview table)
- 4. Member 1: Knowledge Base
- 5. Member 2: Benchmark and Evaluation
- 6. Member 3: Code Analysis and Retrieval
- 7. Member 4: LLM Reasoner and Citation Validator
- 8. Member 5: Fixing, Web App and CLI
- 9. Results everyone must know
- 10. Paper plan and timeline

1. The project in one minute

Problem. When a Django project moves from version 4.0 to 5.x, some functions, arguments and settings it uses have been removed or changed. The code breaks only when it runs. Existing rule tools (django-upgrade) know only a few hand-written fixes, and LLMs like ChatGPT often make up problems that do not exist.

Our idea. CodeMigrate reads your code, finds every Django API you use, looks up the **official Django release notes** for that API, lets an LLM decide if it breaks, and then **checks the LLM's answer with plain code**. A warning is shown only if a real release note backs it. Finally it gives back the **fixed file**.

Easy analogy. Think of a student (the LLM) writing an exam answer with an open book (the release notes). A strict teacher (our citation validator) checks that every answer quotes the right page of the book. Answers without a correct page number get zero.

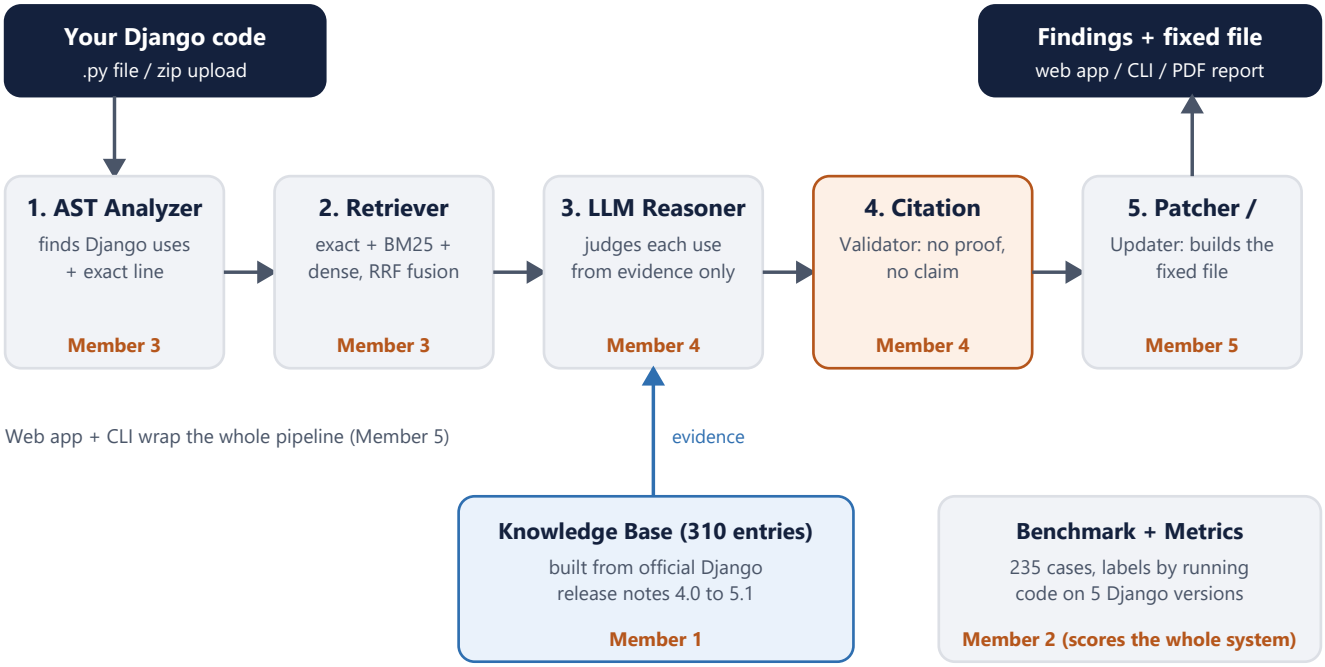
Result in one line. Our system: F1 0.82 with 0% made-up findings. The existing tool django-upgrade: F1 0.02. An LLM alone: F1 0.71 with 29% made-up findings.

Key words

Word	Simple meaning
AST	Abstract Syntax Tree: Python's own tree view of code. Lets us find exact uses of names, even with import aliases, without running the code.
Knowledge Base (KB)	Our database of 310 Django API changes, each linked to the official release-note page.
RAG	Retrieval-Augmented Generation: give the LLM the relevant documents before it answers.
BM25 / dense / RRF	Three ways to search the KB (keywords, meaning, and a method to merge the rankings).
Hallucination	A finding the LLM made up: no real release note supports it.
Precision / Recall / F1	Precision = how many warnings were right. Recall = how many real problems we caught. F1 = balance of both (0 to 1).
Execution labels	The 'answer key': we ran every test case on 5 Django versions and recorded which lines really break.

2. Architecture

The pipeline has five stages. Each code file flows left to right. The Knowledge Base feeds evidence into the retriever. The benchmark measures the whole thing. The web app and CLI are two ways to run it.



Where it lives in the code

Stage	Folder / file	Owner
Build Knowledge Base	src/codemigrate/kb/ (build_kb.py, symbol_index.py, schema.py)	M1
Benchmark + labels	benchmark/ (build_benchmark.py, label_by_execution.py)	M2
Metrics + experiments	evaluation/ (metrics.py, run_experiments.py, plots.py)	M2
1. AST Analyzer	src/codemigrate/analysis/ (ast_analyzer.py, repo_scanner.py)	M3
2. Retriever	src/codemigrate/retrieval/ (exact, bm25, dense, hybrid)	M3
3. LLM Reasoner	src/codemigrate/llm/ (reasoner, prompts, gemini, offline, cache)	M4
4. Citation Validator	src/codemigrate/validation/ (citation_validator.py, matching.py)	M4
Glue: runs stages 1-5	src/codemigrate/pipeline.py	M4
5. Patcher / Updater	src/codemigrate/patching/ (fixer.py, updater.py)	M5
Web app + CLI	src/codemigrate/api/, frontend/src/, src/codemigrate/cli.py	M5

One example, end to end

Input file `views.py`, upgrading 4.0 to 5.1:

```
from django.utils.timezone import utc      # line 1
now = datetime.now(tz=utc)                 # line 2
```

- **Analyzer** sees line 1 imports `django.utils.timezone.utc` and line 2 uses it.
- **Retriever** finds KB entry `django-4.1-deprecated-django_utils_timezone_utc` (deprecated in 4.1, replacement `datetime.timezone.utc`, link to the 4.1 release notes).
- **LLM Reasoner** gets the code plus that entry and answers in JSON: line 1 breaks, cite that id.

- **Validator** checks: the id exists, was retrieved for that line, names the same symbol, and 4.1 is inside 4.0 to 5.1. All true, so the finding is **VERIFIED**.
- **Patcher** rewrites the import to use `datetime.timezone.utc` and returns the whole updated file, with a diff.

3. Task split for 5 members

Each member owns one part: they explain it in the review, answer questions on it, show it in the demo, and write its section of the paper.

Member	Owns	Demo part	Paper section
M1	Knowledge Base from release notes	Open a KB entry and its official link	Knowledge Base construction + Related Work
M2	Benchmark, execution labels, metrics, experiments	Show results table and graphs	Benchmark + Experimental Results
M3	AST analyzer + retrieval (exact, BM25, dense, hybrid)	Show findings with exact lines	Method: analysis and retrieval + ablations
M4	LLM reasoner, citation validator, pipeline	Show a rejected (unsupported) claim	Abstract, Introduction, Method: validator
M5	Patcher/updater, web app, CLI, tests/CI	Run the live scan, Updated code tab	System implementation, Limitations, Conclusion

Review talk order (about 12 minutes)

Time	Who	Slides
0:00-1:30	M4	Title, problem, goal
1:30-3:30	M3 + M4	Pipeline (M3: stages 1-2, M4: stages 3-4)
3:30-5:00	M1	Knowledge base
5:00-7:30	M2	Benchmark, systems A/B/C/D, results, key result, ablation
7:30-8:30	M1	Novelty table
8:30-11:00	M5	Live demo
11:00-12:00	M5 then all	Limits, team slide, questions

MEMBER 1

Knowledge Base (KB)

You own: turning Django's official release notes into a clean, searchable list of 310 API changes.

Explain it simply

The release notes are long web pages written for humans. We turn them into a table the computer can search: one row per change, saying which API, which version, what happened (removed, deprecated, renamed, behaviour changed), what to use instead, and the link to the official page. Every finding CodeMigrate shows points to one of these rows, so every claim can be traced back to Django's own documentation.

How it works

- Download `docs/releases/<version>.txt` from Django's repository at a fixed commit (cached in `data/kb/raw/`), so the build is repeatable.
- Read only three sections: Backwards incompatible changes, Features deprecated, Features removed.
- Turn every bullet into one or more entries. Notes often say a short name like `ModelAdmin.lookup_allowed`; the **symbol index** (an AST scan of Django's own source) maps it to the full dotted path.
- Apply `data/kb/manual_corrections.yaml`: fixes for parser mistakes. Every override is logged and the entry is marked `manually_corrected`.
- Write `data/kb/django_changes.jsonl` (the KB) and `build_report.json`.

Files to know

- `src/codemigrate/kb/build_kb.py`: the build steps above
- `src/codemigrate/kb/schema.py`: what one entry looks like (KBEntry)
- `src/codemigrate/kb/symbol_index.py`: short name to full path
- `src/codemigrate/kb/versions.py`: version comparison helpers
- `data/kb/django_changes.jsonl`: the finished KB

Code walkthrough

```
class KBEntry(BaseModel):          # src/codemigrate/kb/schema.py (simplified)
    id: str                        # e.g. django-4.1-deprecated-django_utils_timezone_utc
    version_introduced: str        # "4.1"
    change_type: ChangeType        # removed | deprecated | behavior_changed | renamed
    affected_api: str              # "django.utils.timezone.utc"
    replacement: str | None        # "datetime.timezone.utc"
    source_url: HttpUrl            # must be an official docs.djangoproject.com link
    provenance: Provenance         # auto_parsed | manually_corrected | manual
    mechanical_fix: bool           # True if the patcher can fix it automatically
```

Pydantic checks every entry when it is loaded: a wrong version format or a non-official link is rejected, so a bad row can never reach the pipeline.

Numbers to remember

- 310 KB entries, Django 4.0 to 5.1 release notes.
- 4 change types: removed, deprecated, renamed, behavior_changed.
- Every entry has an official source_url and a provenance flag.

Likely reviewer questions

Q: Why not just let the LLM remember Django changes?

A: LLM memory can be outdated or wrong, and it cannot give a source. Our KB is built from the official notes, so every finding has a link the user can check.

Q: What if the parser reads a bullet wrongly?

A: We fix it in manual_corrections.yaml. Each fix is logged and the entry is marked manually_corrected, so we can say exactly how many entries were hand-fixed.

Q: Why do you need a symbol index?

A: Release notes use short names. Code imports full paths. The index maps one to the other by reading Django's source code with the AST, without importing or running it.

Q: Can this work for other libraries?

A: The design can, as long as the library has structured release notes. We only built and tested it for Django.

Your paper section

Knowledge Base construction (sources, parsing, symbol resolution, manual corrections) and the Related Work section (read the papers in docs/RELATED_WORK.md first).

MEMBER 2

Benchmark and Evaluation

You own: the test cases, the answer key (labels), the metrics and all result tables and graphs.

Explain it simply

To prove our tool works we need an exam with an answer key. We wrote 235 small Django code cases. To get the answer key **without trusting any tool**, we actually **ran** every case on five Django versions and recorded which lines crash or give a removal warning. Then we score each system's warnings against that key.

How it works

- **Build cases** (build_benchmark.py): synthetic (one KB API placed in small realistic code), negative (clean code and near-misses that should NOT be flagged) and real (files from Django 4.0-era open-source packages).
- **Split** into dev and test by hashing a group key, so variants of the same API never appear in both. All reported results are on the **test** split (160 cases).
- **Label by running** (label_by_execution.py): run each case on Django 4.0.10, 4.1.13, 4.2.30, 5.0.14 and 5.1.15 in a Docker image.
- **Score** (metrics.py): match each warning to a labelled line, count TP / FP / FN, compute precision, recall, F1 with 95% bootstrap confidence intervals, and the hallucination rate.
- **Run everything** (run_experiments.py) and draw figures (plots.py) into results/.

Files to know

- `benchmark/build_benchmark.py`, `benchmark/label_by_execution.py`
- `evaluation/metrics.py`, `evaluation/run_experiments.py`, `evaluation/plots.py`, `evaluation/error_analysis.py`
- `results/tables/*.csv|.tex`, `results/figures/*.png`

Code walkthrough

```
Labeling rule (from benchmark/label_by_execution.py, upgrade 4.0 -> 5.1):
  positive line = a statement that raises an error under some target version (4.1..5.1)
                  but not under 4.0,
                OR emits a Django removal warning (RemovedInDjangoXYWarning, X.Y > 4.0).
  ignored line  = fails already under 4.0, a NameError caused by a failed import,
                  or an error raised deep inside another third-party module.
  scored lines  = lines executed under 4.0 plus labelled lines.

Metric (evaluation/metrics.py):
  TP = labelled lines our findings hit      FN = labelled lines we missed
  FP = scored findings that hit nothing     F1 = 2PR / (P + R)
  hallucinated = cited evidence does not support the claimed symbol / version range
```

The labels never use CodeMigrate itself, so the benchmark cannot be biased toward our tool.

Numbers to remember

- 235 usable cases: dev 75, test 160.
- 5 Django versions run per case.
- Test results: ours (D) F1 0.82, LLM-only (B) 0.71, django-upgrade 0.02, pyupgrade 0.00.
- Hallucination: B 29%, C 6%, D 0%.

Likely reviewer questions

Q: Why not label the cases by hand?

A: Hand labels can be wrong or biased. Running the code on each Django version shows what really breaks.

Q: What is a confidence interval here?

A: We resample the test cases many times (bootstrap) and recompute F1. The 95% range shows how much the score could move with a different sample of cases.

Q: Why is C's F1 (0.83) slightly higher than D's (0.82)?

A: The validator also throws away a few correct findings (recall 0.92 to 0.88), but it removes every unsupported one. We trade a little recall for zero made-up findings.

Q: Why only one Gemini run?

A: The free API tier allows about 500 requests a day. Every call is cached in .cache/llm, so the reported numbers reproduce exactly without new calls.

Your paper section

Benchmark (case types, split, execution labelling, metrics) and Experimental Results (tables, figures, error analysis).

MEMBER 3

Code Analysis and Retrieval

You own: finding every Django API used in the code (with exact line numbers) and searching the KB for it.

Explain it simply

Step one is like highlighting every Django word in an essay, including nicknames (import aliases). Step two is like looking each highlighted word up in the index of a book: we use three different search methods and combine their rankings, so we find the right release note even when the wording is different.

How it works

- **AST analyzer** parses each file into a syntax tree and resolves `import a.b as c`, `from a import b as c`, attribute chains, `settings.X`, dotted strings like `"django.middleware.x.Y"`, Meta options and distinctive method calls.
- Generic names (and dunder methods like `__str__`) are not flagged on unknown objects, to avoid false alarms.
- **Exact retriever**: matches the full dotted name or alias.
- **BM25**: keyword search over the entry text.
- **Dense**: MiniLM sentence embeddings + FAISS index, finds entries with similar meaning.
- **Hybrid**: merges the three ranked lists with Reciprocal Rank Fusion (RRF) and keeps the top-k entries inside the upgrade's version range.

Files to know

- `src/codemigrate/analysis/ast_analyzer.py`, `repo_scanner.py` (reads files/zips)
- `src/codemigrate/retrieval/exact.py`, `bm25.py`, `dense.py`, `hybrid.py`
- `evaluation/retrieval_bench.py` (retrieval quality)

Code walkthrough

```
# src/codemigrate/retrieval/hybrid.py  (idea)
# score(entry) = sum over retrievers r of   w_r / (k_rrf + rank_r(entry))     with k_rrf = 60
#
# Example with all weights = 1: an entry ranked 1st by exact and 3rd by BM25:
#   1/(60+1) + 1/(60+3) = 0.0164 + 0.0159 = 0.0323   -> high final score
# An entry found by only one retriever at rank 10:
#   1/(60+10) = 0.0143                                -> lower
```

RRF only uses ranks, not raw scores, so the three very different scoring methods can be combined fairly without tuning.

Numbers to remember

- Ablation on test (offline reasoner): exact / BM25 / hybrid F1 0.81, dense F1 0.83.
- top-k 1, 3, 5 gave the same result offline.
- Removing dunder methods from 'distinctive' names raised test F1 from 0.76 to 0.79 (found during error analysis, confirmed on dev).

Likely reviewer questions

Q: Why AST and not regex/grep?

A: AST understands Python: it follows import aliases and attribute chains and gives exact line numbers. Regex would miss aliases and match comments or strings by mistake.

Q: Why three retrievers?

A: Exact is precise but misses reworded names; BM25 catches keywords; dense catches meaning. Fusing them is more robust than any single one.

Q: Dense scored best. Why keep hybrid as default?

A: The difference (0.83 vs 0.81) is inside the confidence interval, and hybrid is more robust when names are unusual. We report the ablation honestly.

Q: How do you avoid flagging every .save() or .get()?

A: Common method names on objects whose type we do not know are skipped; only distinctive names are checked.

Your paper section

Method: static analysis and retrieval, plus the retriever and top-k ablation results.

MEMBER 4

LLM Reasoner and Citation Validator

You own: asking the LLM to decide which uses break, and the code that checks every answer (our main novelty).

Explain it simply

The LLM is the student, the release notes are the open book, and the validator is the strict teacher. The student must quote an entry id for every claim. The teacher checks that the quoted entry really exists, was really in front of the student for that line, talks about the same API, and belongs to the right Django versions. Wrong or missing quotes: the claim is thrown out.

How it works

- The pipeline (pipeline.py) runs the analyzer, gets evidence for each code reference, and sends one prompt per file to the LLM.
- Prompt rule: use **only the given evidence**, answer in **JSON**, cite entry ids. Temperature 0.
- Providers: Gemini (gemini-3.5-flash-lite, free tier), OpenAI-compatible, and an **offline** rule-based reasoner that needs no API key.
- Every LLM call is cached on disk by a hash of (provider, model, prompt, settings), so reruns are free and identical.
- The validator marks each finding VERIFIED, UNVERIFIED or UNSUPPORTED_CLAIM and logs the reason.
- Systems compared: B = LLM alone; C = LLM + evidence; D = LLM + evidence + validator (ours).

Files to know

- `src/codemigrate/pipeline.py` (scan_sources: the main loop)
- `src/codemigrate/llm/`: reasoner.py, prompts.py, gemini.py, openai.py, offline.py, cache.py, base.py (retries, quota handling)
- `src/codemigrate/validation/citation_validator.py`, `matching.py`

Code walkthrough

```
# src/codemigrate/validation/citation_validator.py  -- "no proof, no claim"
# A finding is VERIFIED only if at least one cited id:
#   1. exists in the KB,
#   2. was retrieved as evidence for a code reference on the claimed line,
#   3. is within the requested version range (e.g. 4.0 -> 5.1),
#   4. names the claimed symbol and the code reference's symbol.
# No valid citation (none, unknown id, or not in that line's evidence) -> UNSUPPORTED_CLAIM
# Citation exists but fails line / symbol / version checks          -> UNVERIFIED

# src/codemigrate/pipeline.py  (core of scan_sources, simplified)
for ref in file.refs:
    hits = retriever.search(ref, top_k, from_version, to_version)
    if hits: evidences.append(Evidence(ref, hits))
findings = reason_grounded(client, path, source, evidences, ...)
for f in findings:
    vf = validator.validate(f, evidences)
    (result.findings if vf.status is VERIFIED else result.rejected).append(vf)
```

The validator is plain deterministic Python: no LLM is used to check the LLM.

Numbers to remember

- Hallucination: B 29% -> C 6% -> D 0%.
- D F1 0.82, precision 0.78, recall 0.88.
- C and D share the same LLM answers (same cache), so the only difference is the validator.

Likely reviewer questions

Q: What is new compared to normal RAG?

A: Normal RAG puts documents in the prompt and trusts the answer. We also check the answer against the documents with code, and drop anything unsupported.

Q: Why not ask a second LLM to check the first?

A: A second LLM can also be wrong. Our checks are exact rules (ids, lines, symbols, versions), so they are repeatable and explainable.

Q: What does the offline reasoner do?

A: It applies fixed rules to the retrieved evidence instead of calling an LLM. It reaches F1 0.81 and lets anyone run the tool without an API key.

Q: Why temperature 0 and a cache?

A: To make results repeatable: the same input gives the same output, and cached calls cost nothing.

Your paper section

Abstract and Introduction (problem, idea, contributions, headline numbers) and Method: the LLM reasoner and the citation validator.

MEMBER 5

Fixing, Web App and CLI

You own: turning verified findings into a fixed file, and the user-facing app, command line, tests and CI.

Explain it simply

Finding problems is only half the job; users want the corrected code. We only change code we are sure about: simple renames and moves are fixed automatically, harder cases can be rewritten by the LLM but only if the rewrite passes strict checks, and anything else gets a clear TODO comment. Users do all this from a web page or one command.

How it works

- **Fixer** (fixer.py): for KB entries marked mechanical_fix, makes a unified diff for same-place renames (assertFormsetError to assertFormSetError) and module moves (django.utils.timezone.utc to datetime.timezone.utc). Every patch must still parse. Rule: no patch beats a wrong patch.
- **Updater** (updater.py + pipeline.update_files): applies safe patches bottom-up, then optionally asks the LLM to rewrite the rest. The rewrite is accepted only if it parses, stays at least 70% similar to the original, and a re-scan finds no new problems. Otherwise a `# TODO(codemigrate): ...` comment is added (comments never change program behaviour).
- **API** (FastAPI): register/login with JWT, projects, scans run in a background thread pool (PENDING to RUNNING to COMPLETED or FAILED), Markdown and PDF reports.
- **Frontend** (React + Vite + TypeScript): Login, Dashboard (paste code or upload zip, choose versions, reasoner, retriever), Scan view with tabs Original / Updated code / Changes, History, Compare.
- **CLI**: `codemigrate scan ... --write-fixed DIR` writes fixed copies (never touches the originals).
- **Quality**: 52 pytest tests, 72% coverage (gate 70%), Playwright end-to-end test, GitHub Actions CI.

Files to know

- `src/codemigrate/patching/fixer.py`, `updater.py`
- `src/codemigrate/api/`: main.py, routes.py, auth.py, jobs.py, db.py, reports.py
- `frontend/src/pages/`: Dashboard.tsx, ScanView.tsx, History.tsx, Compare.tsx
- `src/codemigrate/cli.py`, `.github/workflows/ci.yml`

Code walkthrough

```
# API routes (src/codemigrate/api/routes.py)
POST /auth/register   POST /auth/token   GET /auth/me
POST /projects        GET  /projects
POST /projects/{pid}/scans          (code in JSON)   -> 202, scan runs in background
POST /projects/{pid}/scans/upload   (zip file)
GET  /scans/{sid}                   findings + updated files
GET  /scans/{sid}/report.md   GET /scans/{sid}/report.pdf
GET  /results?split=test            benchmark tables for the Compare page

# Run locally
.venv\Scripts\python -m uvicorn codemigrate.api.main:app --port 8000
cd frontend; npm run dev            -> http://localhost:5173
```

Use the offline reasoner in the live demo: it needs no API key and cannot run out of quota.

Numbers to remember

- 52 tests pass, 72% coverage.
- Scan states: PENDING, RUNNING, COMPLETED, FAILED.
- LLM rewrite accepted only if similarity is at least 0.7 and the re-scan is clean.

Likely reviewer questions

Q: Can the automatic fix break my code?

A: Mechanical patches only cover exact renames and moves, and every patch must parse. LLM rewrites are re-checked; if a check fails we add a TODO comment instead of changing code.

Q: Why run scans in the background?

A: An LLM scan can take a while. The API returns at once (202) and the page polls the status.

Q: How are users kept apart?

A: JWT login; every project and scan query checks the owner, so users only see their own data.

Q: Why no Docker?

A: It is optional (a docker-compose file exists), but the app runs directly with Python and Node, which is simpler for the demo.

Your paper section

System implementation (API, UI, updated-code feature, testing), Limitations and Conclusion.

9. Results everyone must know

Test split, 160 cases. Gemini rows are one run (seed 0, temperature 0); 95% bootstrap confidence intervals in brackets.

System	Precision	Recall	F1 [95% CI]	Made-up
A: django-upgrade	0.09	0.01	0.02 [0.00, 0.06]	n/a
A: pyupgrade	0.00	0.00	0.00	n/a
B: LLM only (Gemini)	0.68	0.74	0.71 [0.63, 0.78]	29%
C: LLM + release notes	0.77	0.92	0.83 [0.77, 0.89]	6%
D: offline, no LLM	0.75	0.89	0.81 [0.74, 0.88]	0%
D: full pipeline (ours)	0.78	0.88	0.82 [0.75, 0.88]	0%

Three sentences every member should be able to say:

- Existing rule tools catch almost none of these breaking changes (F1 0.02).
- Giving the LLM the release notes cuts made-up findings from 29% to 6%; our validator removes the rest (0%) at a small recall cost.
- The answer key comes from actually running the code on five Django versions, not from any tool.

Honest limits (say them before you are asked)

- Django only, versions 4.0 to 5.1.
- One LLM, one run, because of the free-tier quota.
- Benchmark cases are small files, not whole projects.
- Published numbers from other papers are on different tasks, so we compare head-to-head only with django-upgrade and pyupgrade, plus our own ablations.

10. Paper plan and timeline

The LaTeX skeleton is in `paper/main.tex` (IEEE format) with an outline in each section; tables and figures are already linked from `results/`. References are in `paper/refs.bib`.

Section	Writer	Build it from
Abstract, Introduction	M4	Problem + goal, contributions, headline numbers
Related Work	M1	docs/RELATED_WORK.md (read each paper first)
Knowledge Base	M1	kb/ code, build_report.json
Method: analysis + retrieval	M3	analysis/, retrieval/
Method: reasoner + validator	M4	llm/, validation/, pipeline.py
Benchmark	M2	benchmark/ docstrings, dataset card
Experiments + Results	M2 (M3 for ablations)	results/tables, results/figures, error_cases.md
System implementation	M5	api/, frontend/, patching/
Limitations, Conclusion	M5	Limits list above

Rules for writing

- Write in your own words from the code and results. Do not paste from this guide, the slides, papers or any AI tool, and do not use paraphrasing tools (they get flagged as AI).
- Cite only papers you have read. Check each BibTeX entry on the publisher page.
- Run your similarity and AI checks per section early, not just at the end.
- Every number in the paper must match results/tables. Ask for a fact-check of each draft.

Suggested timeline

When	What
Days 1-2	Each member reads their files and this guide; practise explaining your part in 2 minutes.
Days 3-4	Rehearse the review talk and live demo once together; record a backup demo video.
Days 5-8	Each member drafts their paper section.
Days 9-10	Merge in Overleaf; fact-check numbers; run similarity and AI checks.
Day 11+	Fix, final read-through, fill author names, submit.